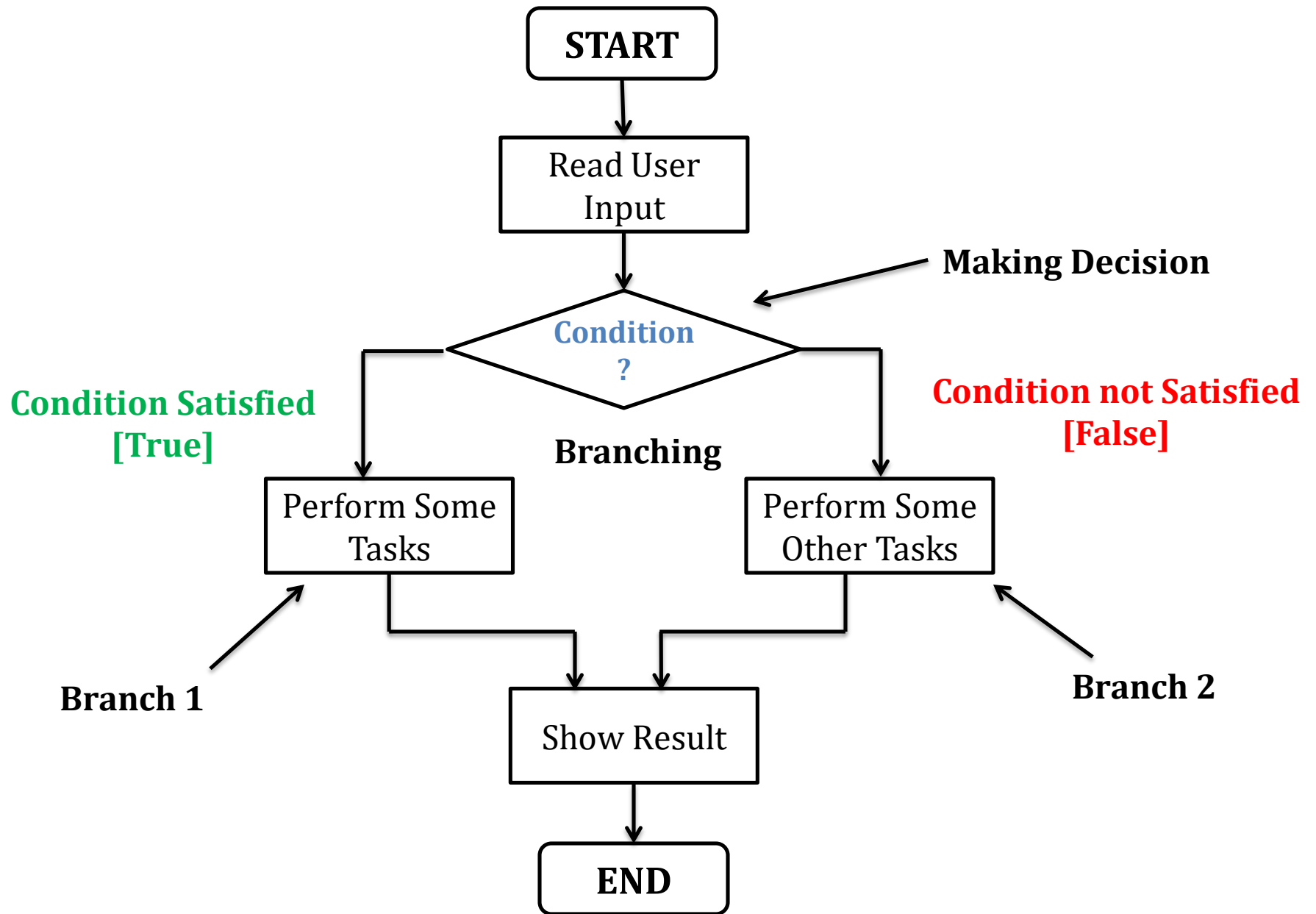


Decision Making and Branching

Introduction

- **“Decision making and branching”** is one of the most important concepts of computer programming.
- Programs should be able to make logical (true/false) decisions based on the condition provided.
- Every program has one or few problems to solve. In order to solve those particular problems important decisions have to be made depending on the nature of the problems.
- Generally C program execute it's statements sequentially. But in order to solve problems we may have some situations where we have to change the order of executing the statements based on whether some conditions have met or not. So controlling the execution of statements based on certain condition or decision is called decision making and branching.



Decision Making & Branching in C

- C language possesses decision making and branching capabilities by supporting the following statements:
 1. If statement
 2. Switch statement
 3. Conditional operator statement
 4. goto statement
- These statements are known as **decision making statements**. They are also called **control statements** as they control the flow of execution.

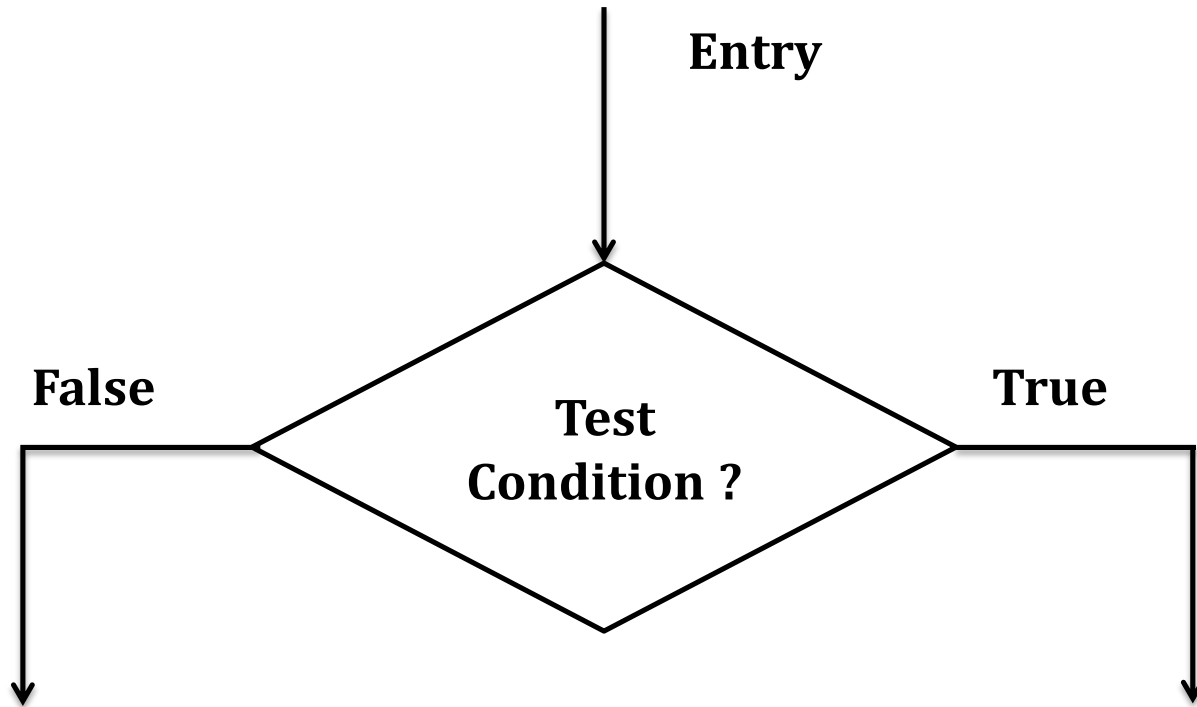
IF Statement

- The if statement is a powerful statement for decision making and is used to control the flow of execution of statements. **It is basically a two-way decision making statement and is used in conjunction with an expression.** It takes the following structure:

if (test-condition)

- It allows the computer to evaluate the expression first and then depending on whether the value of the expression or condition is true or false, it transfer the control to a particular statement. This point of program has two paths to follow, one for the true condition and the other for the false condition.

IF Statement



Example:
If(Pocket balance is zero)
 Borrow money;

IF Statement

- The if statement can be implemented in four different forms depending on the complexity of the conditions to be tested. The four forms are:
 1. Simple if statement
 2. If else statement
 3. Nested if else statement
 4. Else if ladder

Simple if Statement- Structure

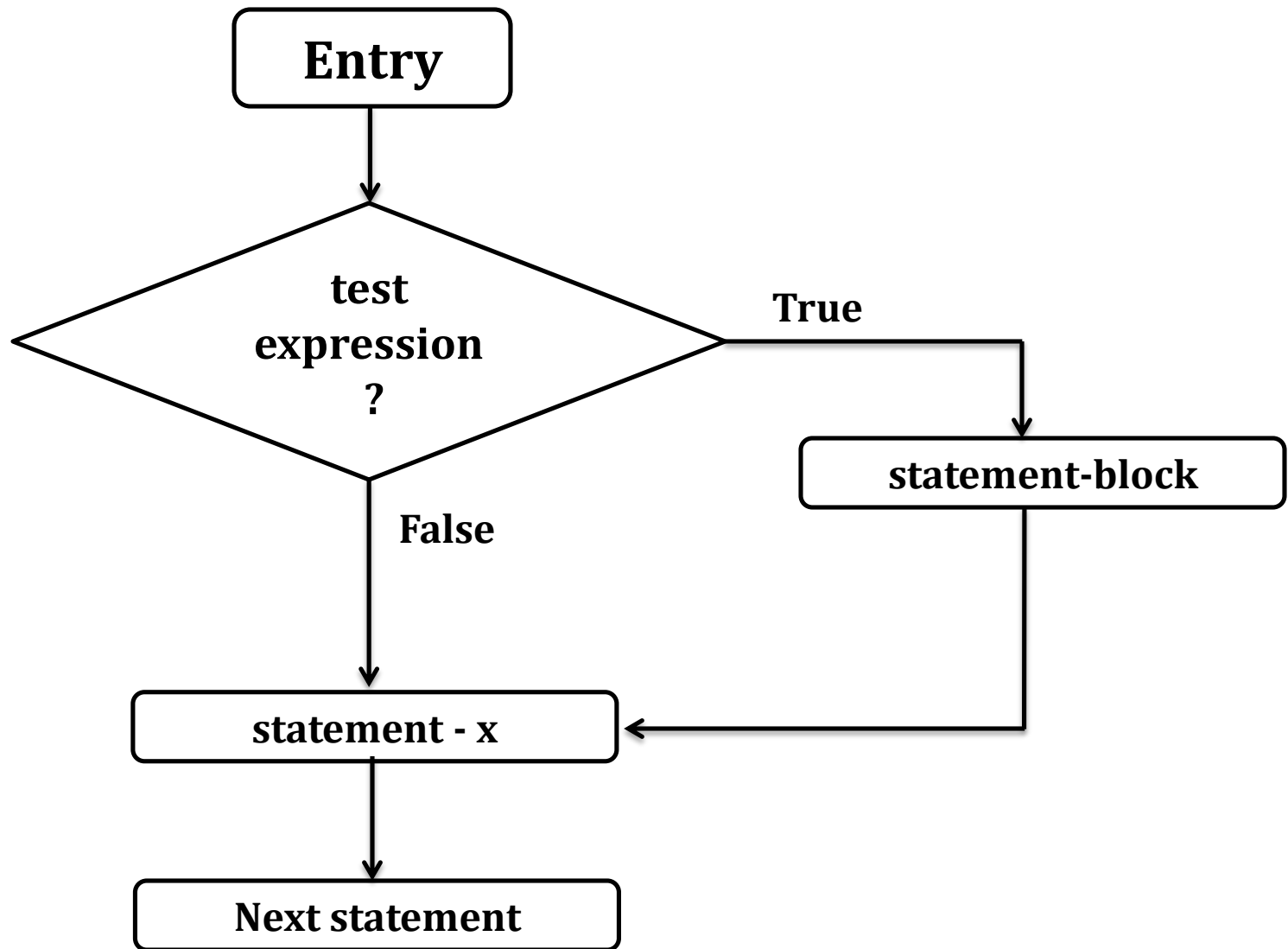
- The general form of a simple if statement is:

```
if (test_condition)
{
    statement-block;
}
statement x;
```


Simple if Statement

- The statement block may consists of a single statement or a group of statements.
- If the test_condition is satisfied then the statement block will be executed first then the statement-x will be executed after completing the execution of statement block.
- Otherwise if the test_condition doesn't satisfied then, the statement block will be skipped and the execution will jump to the statement-x.
- So in short when the condition is true then both the statement block and the statement-x are executed but in sequence.

Simple if Statement- Flowchart



Simple if Statement

- Important things to notice that if you want to control a single statement using the if condition then no need to create a block using the curly braces.
- The curly braces is used to create statement block when it is necessary to include multiple statements under the same if condition.

Simple if Statement-Example

- Example with **block of statements**:

```
if (marks >= 90)
```

```
{  
  marks = marks + bonus_marks;  
  grade = "A+";  
}
```

**Condition controlled
statement**

```
printf("The mark achieved: %d" , marks);
```

Simple if Statement

- Example of **single statement**:

```
if (marks >= 90)
```

```
    marks = marks + bonus_marks; ← Condition controlled  
statement
```

```
printf("The mark achieved: %d", marks);
```

Simple if Statement

- **What happens if multiple statements follows after the if condition without using the curly brace?**
- Answer: Then the statement immediately following the if condition is considered as the controlling statement of the if condition. if the condition is true the controlled statement is executed otherwise not. Rest of the statements are executed sequentially.

Simple if Statement

- Example:

```
if (marks >= 90)
```

```
    marks = marks + bonus_marks; ← Condition controlled  
                                statement
```

```
    grade = "A+";
```

```
printf("The mark achieved: %d ", marks);
```

if else Statement

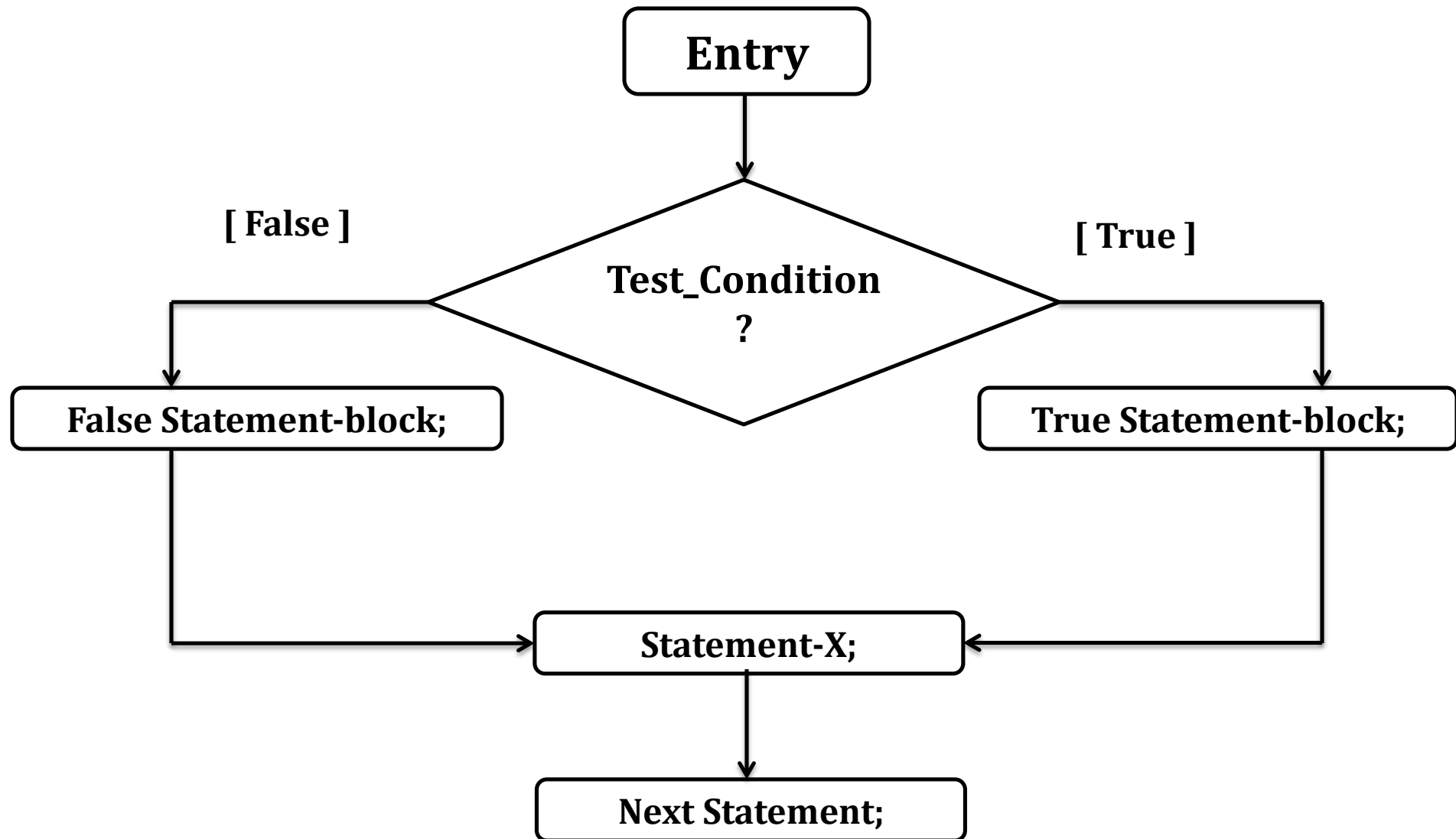
- If the test condition is true then the true block statements, immediately following the if statements are executed;
- Otherwise the false block statements are executed.
- In short either true-block or false-block of statements will be executed, not both.
- But in both cases the control is transferred subsequently to the statement-x as it is an independent (not controlled by the if else statement) statement.
- **It is also called two way conditional branching**

if else Statement- Structure

- The if else statement is an extension of the simple if statement. The general form is :

```
if (test_condition)
{
    True block statements;
}
else
{
    False block statements;
}
statement-x;
```

if else Statement- Flowchart



if else Statement- Example

- Example with block of statement:

```
if (marks >= 40)
```

```
{  
    marks = marks + bonus_marks;  
    grade = "passed";  
}
```

True block statement

```
else
```

```
{  
    marks = marks;  
    grade = "failed";  
}
```

False block statement

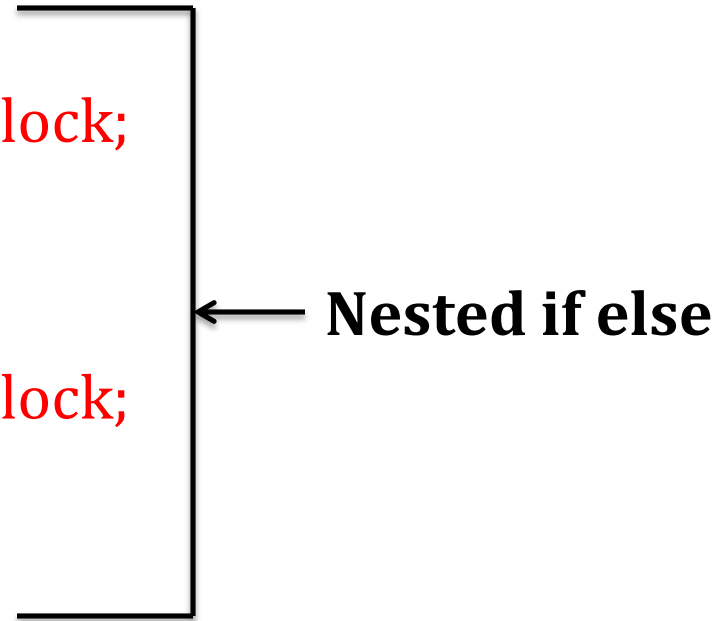
```
printf("The mark achieved: %d ", marks);
```

Nested if else Statement

- Using “if...else statement” within another “if...else statement” is called ‘nested if statement’. “Nested if statements” is mainly used to test multiple conditions.
- **It is called nested conditional branching.**
- They can be structured using following syntax:

Nested if else Statement- Structure

```
if (test_condition)
{
    if (test_condition)
        {
            statement-block;
        }
    else
        {
            statement-block;
        }
}
else
{
    statement-block;
}
statement-x;
```



A diagram illustrating the structure of a nested if-else statement. A large right-facing curly bracket groups the inner if-else block. An arrow points from the text "Nested if else" to the middle of this bracket.

Nested if else -Example

```
if (gender==female)
{
```

```
    if (age<10)
```

```
    {
```

```
        provide free entry;
```

```
        provide free food;
```

```
    }
```

```
    else
```

```
    {
```

```
        provide only free entry;
```

```
    }
```

```
}
```

```
else
```

```
{
```

```
    statement-block;
```

```
}
```

```
statement-x;
```



Nested if else

Else if ladder Statement

- The word ladder means the staircase. As the name implies this statement is used to choose right way/paths among multiple paths.
- There is another way of putting 'if's together when multipath decisions are involved.
- A multipath decision is a chain of 'if's in which the statement associated with an else condition behaves like another if condition.

Else if ladder- Structure

- if (test_condition 1)
 statement-1;
 else if (test_condition 2)
 statement-2;
 else if (test_condition 3)
 statement-3;
 else if (test_condition 4)
 statement-4;

 else if (test_condition n)
 statement-n;
else
 statement-x;

Else if ladder- Example

```
if(Mark>=50 && Mark<60)
{
    printf("Your grade is D");
}
    else if(Mark>=60 && Mark<70)
    {
        printf("Your grade is C \n");
    }
        else if(Mark>=70 && Mark<80)
        {
            printf("Your grade is B \n");
        }
            else if(Mark>=80 && Mark<=100)
            {
                printf("Your grade is A \n");
            }
else
    printf("you have failed");
```

```
if(Mark > 100 || Mark < 0)
{
    printf("Mark is out of range");
}
else
{
    -----
    -----
}
```

Switch Statement

- When one of the many statements is to be selected, then if conditional statement can be used to control the selection.
- However the complexity of such a program increases dramatically when the number of statements increases.
- Fortunately, C has a built in multiway decision making statement known as switch.
- The switch statement tests the value of a given variable or expression against a list of case values and when a match is found only then a block of statements associated with that case is executed.
- The general form is given below:

Switch Statement-Structure

```
switch(expression/ value)
{
    case value-1:
        statement-block-1;
        break;
    case value-2:
        statement-block-2;
        break;
    .....
    case value-n:
        statement-block-n;
        break;
    default:
        default-statement-block;
        break;
}
statement-x;
```

Rules for Switch Statement

- The switch statement must be an integral type.
- Case labels must be constant or constant expression.
- Case labels must be unique. No two labels can have the same value.
- Case labels must end with colon.
- The break statement transfer the control out of the switch statement.
- The break statement is optional. So two or more case labels may belong to the same statements.
- The default label is optional. If present, it will be executed when the expression does not find a matching case label.
- There can be at most one default label.
- The default may be placed any where but usually placed at the end.
- It is permitted to nest switch statements.

Conditional Operator

- The C language has an unusual operator which is useful for making two way decisions.
- This operator is a combination of ? and :
- It takes three operands. This operator is popularly known as the conditional operator.
- The conditional operator can be used as the replacement of if else conditional statement for two way decision making.

Conditional Operator

- The general structure of conditional operator:

Conditional expression? true-statement 1: false-statement;

- The condition is evaluated first. If the result is true then the statement 1 is executed and its value is returned.
- Otherwise statement 2 is executed and its value is returned.
- Example:

flag = (x<0) ? 0 :1;

Conditional Operator

Conditional operator:

```
flag = (x<0) ? 0 :1;
```

It is equivalent to:

```
if(x<0)  
    flag=0;  
else  
    flag=1;
```

